

# Exercise Sheet 4

## Efficient Computing/Parallel Computing 2 in SS 2026

Date of issue: 28.4.2026

Deadline: 4.5.2026

The exercises marked with ★ are relevant for admission to the examination and must be submitted in electronic form in Moodle by the submission deadline.

### Exercise 4.1 (*Introduction to SIMD processing via inline assembly and external kernels*).

In this exercise, we work directly on the Raspberry Pis and not on the Ara cluster. The aim of this exercise is to introduce the compilation of C/C++ source code alongside with optimized assembly kernels. These kernels are either integrated directly into the C/C++ code as *inline assembly* or they are written as separate external kernels and referenced via function prototypes and linking.

Throughout this exercise, we consider two versions of a floating-point fused multiply-add instruction: the scalar version `fmadd` and the vector version `fmla`. More precisely, we are interested in replacing multiple (scalar) instructions `fmadd` by a single (vector) instruction `fmla`. To this end, consider the following corresponding source files.

- `driver.cpp` ⇐: The main program and driver code written in the language C++, which calls all the assembly kernels. (To-do: The two functions `benchmark_inline_asm_1` and `benchmark_inline_asm_2` have to be completed.)
- `benchmark_c.c` ⇐: A code implementing fused multiply-add operations explicitly via the operations `*` and `+`. It serves as a reference and helps with coding.
- `benchmark_asm_1.s` ⇐: The assembly kernel to which the (vector) instruction `fmla` has to be added. (To-do: Insert the code for `fmla`.)
- `benchmark_asm_2.s` ⇐: The assembly kernel containing multiple (scalar) instructions `fmadd`.

There are three tasks.

- a) Edit the file `src/benchmark_asm_1.s` by adding a single `fmla` instruction such that this external assembly kernel performs the same computation as the assembly kernel with multiple `fmadd` instructions given in the file `src/benchmark_asm_2.s`.
- b) Edit the file `src/driver.cpp`. More precisely, given the `fmla` instruction from the previous task, add it to the function `benchmark_inline_asm_1` as inline assembly.
- c) Edit the file `src/driver.cpp`. More specifically, given the assembly kernel from the file `src/benchmark_asm_1.s`, add this kernel including its while loop to the function `benchmark_inline_asm_2` as inline assembly.

The code is compiled on the Raspberry Pis by calling `make` using the `Makefile` ⇐. The executable file is run with the command `./build/exec.out`.

**Exercise 4.2** (*Benchmarking SIMD instructions*).

In this exercise, we work directly on the Raspberry Pi and not on the Ara cluster. Assume that the clock frequency of the Raspberry Pi is 1.5 GHz.

- a) Derive the theoretical peak performance of the Raspberry Pi assuming a full pipeline of `fmla` instructions, each carried out on vectors consisting of 4 floats.
- b) Derive the theoretical peak performance of the Raspberry Pi assuming a full pipeline of `fmadd` instructions, each carried out on scalar float values.

For the following two tasks, take care of read-after-write dependencies, the overhead of control structures, and efficient pipelining. Measure, compute, and print the achieved performance of the following two codes.

- c) Modify `benchmark_asm_1.s` (containing `fmla` instructions) such that its execution time approaches the theoretical peak performance.
- d) Modify `benchmark_asm_2.s` (containing `fmadd` instructions) such that its execution time approaches the theoretical peak performance.

The code can be compiled on the Raspberry Pi by calling `make`. The executable file can be run with `./build/exec.out`